# +Descripció

Treballarem amb una base de dades que conté col·leccions relacionades amb una aplicació d'entreteniment cinematogràfic:

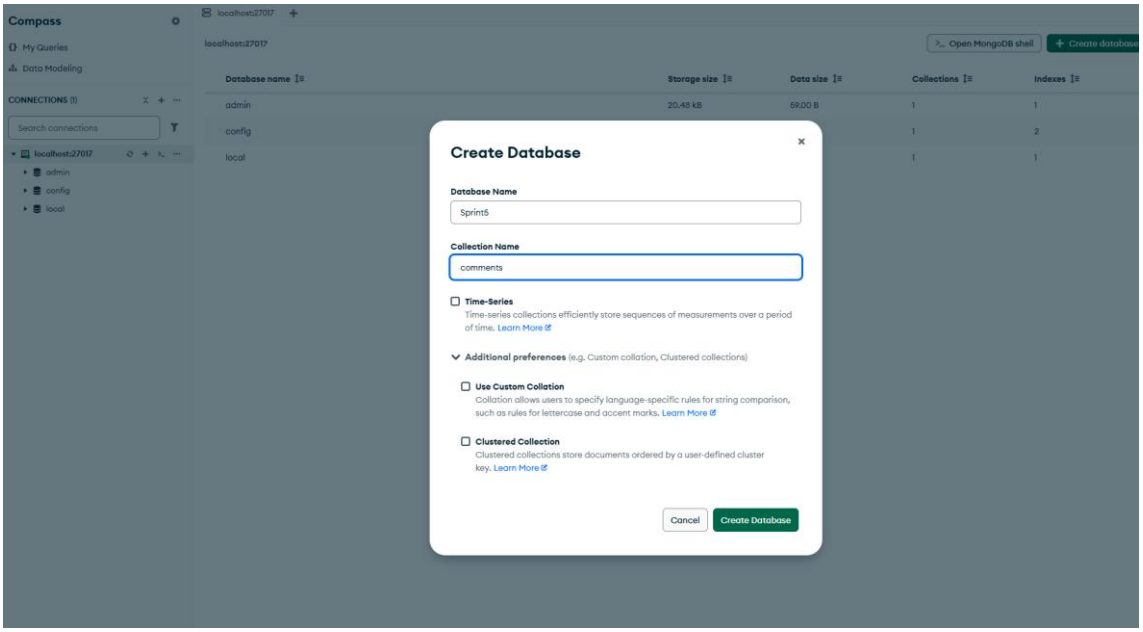
- users: Emmagatzema informació d'usuaris/es, incloent-hi noms, emails i contrasenyes xifrades.
- theatres: Conté dades de cinemes, com ID, ubicació (direcció i coordenades geogràfiques).
- sessions: Guarda sessions d'usuari, incloent-hi ID d'usuari i tokens JWT per a l'autenticació.
- movies: Inclou detalls de pel·lícules, com a trama, gèneres, durada, elenc, comentaris, any de llançament, directors, classificació i premis.
- comments: Emmagatzema comentaris d'usuaris/es sobre pel·lícules, amb informació de l'autor/a del comentari, ID de la pel·lícula, text del comentari i la data.

Duràs a terme algunes consultes que et demana el client/a, el qual està mesurant si seràs capaç o no de fer-te càrrec de la part analítica del projecte vinculat amb la seva base de dades.

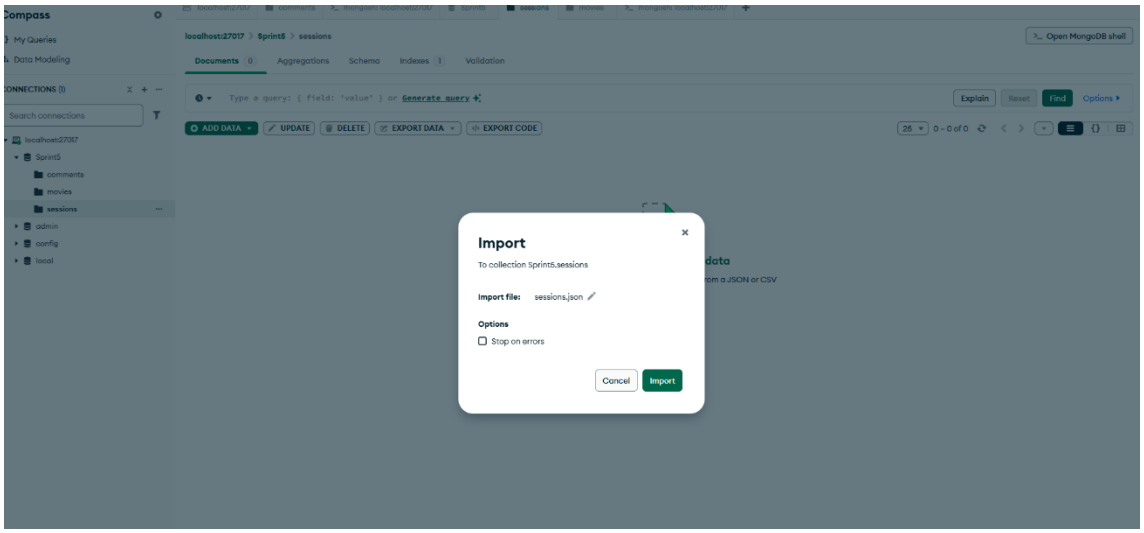
# Nivell 1

**Crea una base de dades amb MongoDB utilitzant com a col·leccions els arxius adjunts.**

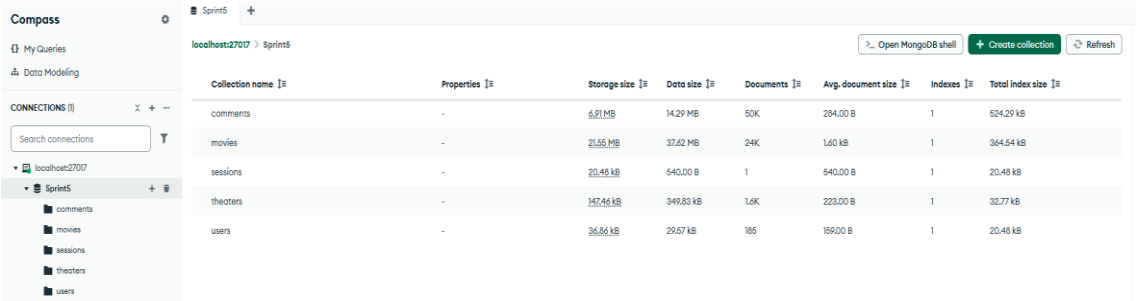
Creem la BBDD amb l'opció "Create Database" de MongoDB Compass, a la qual anomenem Sprint5



Després afegim les cinc col·leccions a la BBDD, amb l'opció "Create Collection" i important els cinc fitxers ***.json




Quedant de la següent manera:




| Collection name | Properties | Storage size | Data size | Documents | Avg. document size | Indexes | Total index size |
|---|---|---|---|---|---|---|---|
| comments | - | 6.91 MB | 14.29 MB | 50K | 284.00 B | 1 | 524.29 kB |
| movies | - | 21.55 MB | 37.62 MB | 24K | 1.60 kB | 1 | 364.54 kB |
| sessions | - | 20.48 kB | 540.00 B | 1 | 540.00 B | 1 | 20.48 kB |
| theaters | - | 147.46 kB | 349.63 kB | 1.6K | 223.00 B | 1 | 32.77 kB |
| users | - | 36.86 kB | 29.57 kB | 185 | 159.00 B | 1 | 20.48 kB |

# Exercici 1

- **Mostra els 2 primers comentaris que hi ha en la base de dades.**

Fem la consulta a través del shell de MongoDB Compass. En aquest cas, amb **find()** sense filtre, retornem tots els comentaris i amb **limit(2)** restringim el resultat als 2 primers.

```
>_MONGOSH
> use Sprint5
< switched to db Sprint5
> db.comments.find().limit(2)
< {
    _id: ObjectId('5a9427648b0beebeb69579cc'),
    name: 'Andrea Le',
    email: 'andrea_le@fakegmail.com',
    movie_id: ObjectId('573a1390f29313caabcd418c'),
    text: 'Rem officiis eaque repellendus amet eos doloribus. Porro dolor voluptatum voluptates neque culpa molestias. Voluptate unde nulla temporibus ullam.',
    date: 2012-03-26T23:20:16.000Z
  }
  {
    _id: ObjectId('5a9427648b0beebeb69579cf'),
    name: 'Greg Powell',
    email: 'greg_powell@fakegmail.com',
    movie_id: ObjectId('573a1390f29313caabcd41b1'),
    text: 'Tenetur dolorum molestiae ea. Eligendi praesentium unde quod porro. Commodi nisi sit placeat rerum vero cupiditate neque. Dolorum nihil vero animi.',
    date: 1987-02-10T00:29:36.000Z
  }
```

- **Quants usuaris tenim registrats?**

Amb **countDocuments()** comptem tots els documents de la col·lecció **users.** Cada document representa un usuari registrat.

```
> db.users.countDocuments()
< 185
```

- **Quants cinemes hi ha en l'estat de Califòrnia?**

Accedim al camp **state** que és un subdocument dins de **location.address**, per comptar a través de **countDocuments()** quants tenen el codi "CA" de Califòrnia

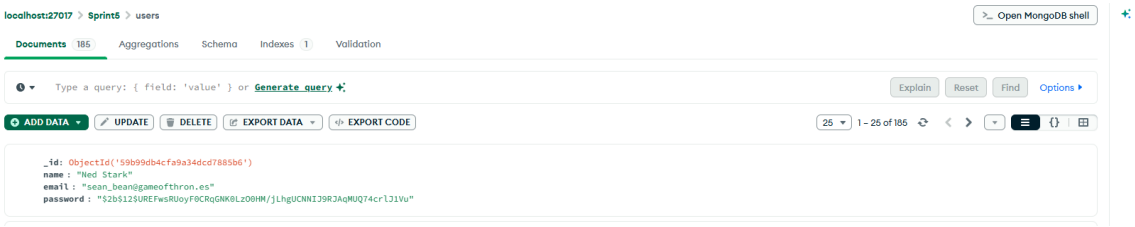
```
> db.theaters.countDocuments({ "location.address.state": "CA" })
< 169
```

- **Quin va ser el primer usuari/ària en registrar-se?**

L'_id de MongoDB (ObjectId) conté un timestamp de creació. Ordenant per **_id: 1 (ascendent)** obtenim el document creat primer. Amb **limit(1)** retornem només el primer usuari en registrar-se.

```
> use Sprint5
< switched to db Sprint5
> db.users.find().sort({_id: 1}).limit(1)
< {
    _id: ObjectId('59b99db4cfa9a34dcd7885b6'),
    name: 'Ned Stark',
    email: 'sean_bean@gameofthron.es',
    password: '$2b$12$UREFwsRUoyF0CRqGNK0LzO0HM/jLhgUCNNIJ9RJAqMUQ74crlJ1Vu'
  }
```

A Compass, sense fer cap modificació també podem veure el primer usuari que es va registrar.




- Quantes pel·lícules de comèdia hi ha en la nostra base de dades?

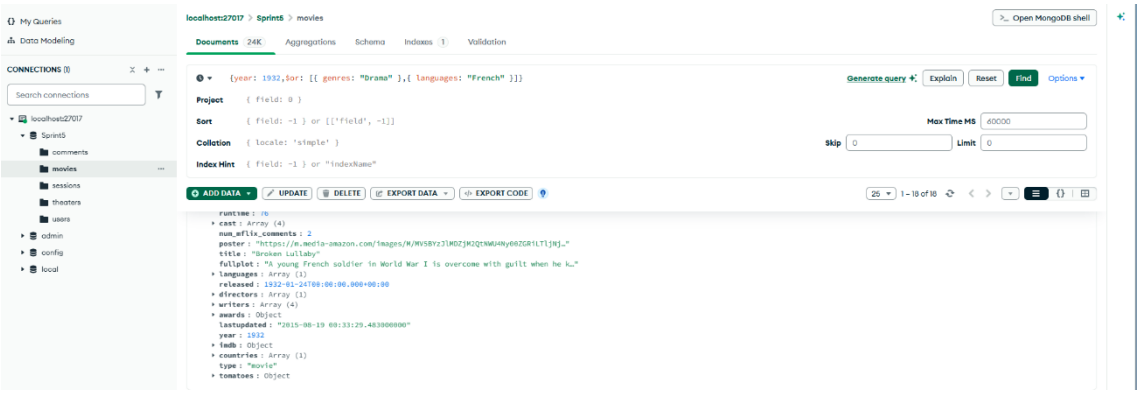
Tornem a fer un **countDocuments()** en el camp genres, que és un vector de la col·lecció "movies". MongoDB cerca dins dels vector automàticament, de manera que **{ genres: "Comedy" }** retorna totes les pel·lícules que continguin "**Comedy**" entre els seus gèneres.

```
>_MONGOSH
> use Sprint5
< switched to db Sprint5
> db.movies.countDocuments({ genres: "Comedy" })
< 7024
```

# Exercici 2

**Mostra'm tots els documents de les pel·lícules produïdes en 1932, però que el gènere sigui drama o estiguin en francès.**

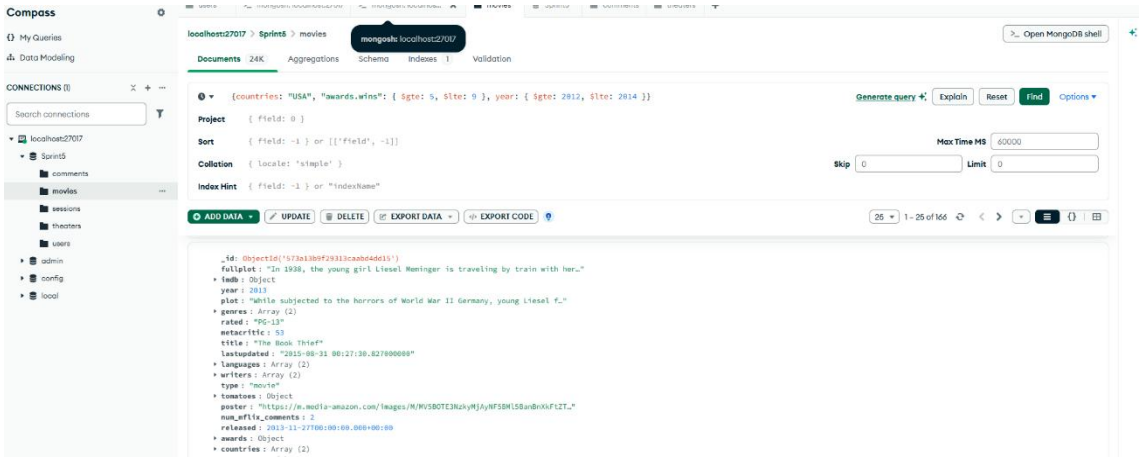
Fem un **$or** amb pel·lícules de l'any 1932 que siguin de drama o estiguin en francès. **year** al ser un camp numèric, no calen cometes. Ho fem directament al Compass amb un **Find** amb resultat de **18**




# Exercici 3

**Mostra'm tots els documents de pel·lícules estatunidenques que tinguin entre 5 i 9 premis que van ser produïdes entre 2012 i 2014.**

Combinem tres filtres: countries: "USA" filtrem les estatunidenques. "**awards.wins**" accedeix al subdocument awards usant **$gte (≥) i $lte (≤)** per definir el rang de 5 a 9 premis. I **year** limita a les produccions entre 2012 i 2014 amb el mateix patró de rang.
Posem la captura del **Find** de Compass per veure que el resultat és **166**. En el Shell, l'estructura de la consulta seria la mateixa.




# Nivell 2

## Exercici 1

**Compte quants comentaris escriu un usuari/ària que utilitza "GAMEOFTHRON.ES" com a domini de correu electrònic.**

En el **shell**, fem un **countDocuments()** accedint al camp email, fent servir una expressió regular amb **$regex**. Amb **$options: "i"** fem que en la cerca no es distingeixi entre **majúscules/minúscules**.

```
> db.comments.countDocuments({ email: { $regex: "@GAMEOFTHRON.ES", $options: "i" }})
< 22841
```

## Exercici 2

**Quants cinemes hi ha en cada codi postal situats dins de l'estat Washington D. C. (DC)?**

En el shell fem un agregació en dues etapes:

1. **$match**: Filtrem només els cinemes de l'estat "**DC**".
2. **$group:** Agrupem els documents pel codi postal (zipcode) i comptem quants n'hi ha a cada un amb **$sum: 1**.

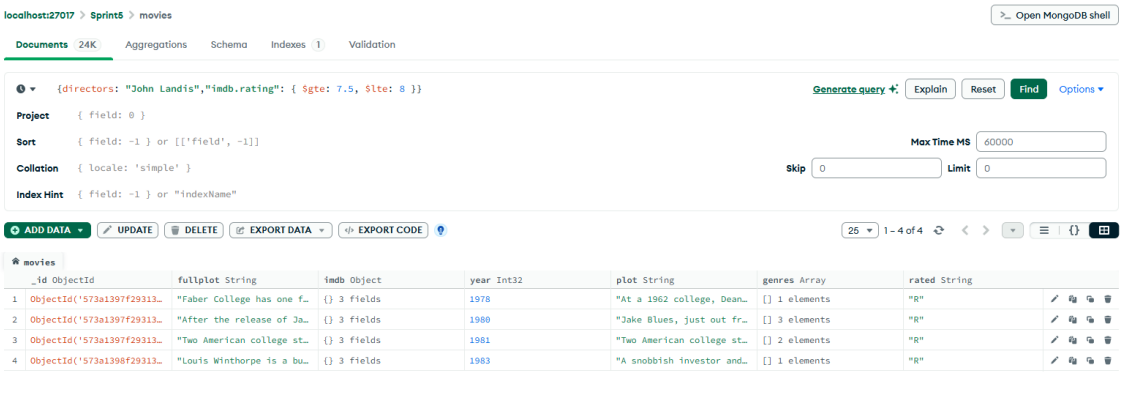
```
> db.theaters.aggregate([{$match: { "location.address.state": "DC" }},
    { $group: { _id: "$location.address.zipcode", total_cinemes: { $sum: 1 }}},])
< {
    _id: '20016',
    total_cinemes: 1
  }
  {
    _id: '20010',
    total_cinemes: 1
  }
  {
    _id: '20002',
    total_cinemes: 1
  }
```

# Nivell 3

## Exercici 1

**Troba totes les pel·lícules dirigides per John Landis amb una puntuació IMDb (Internet Movie Database) d'entre 7,5 i 8.**

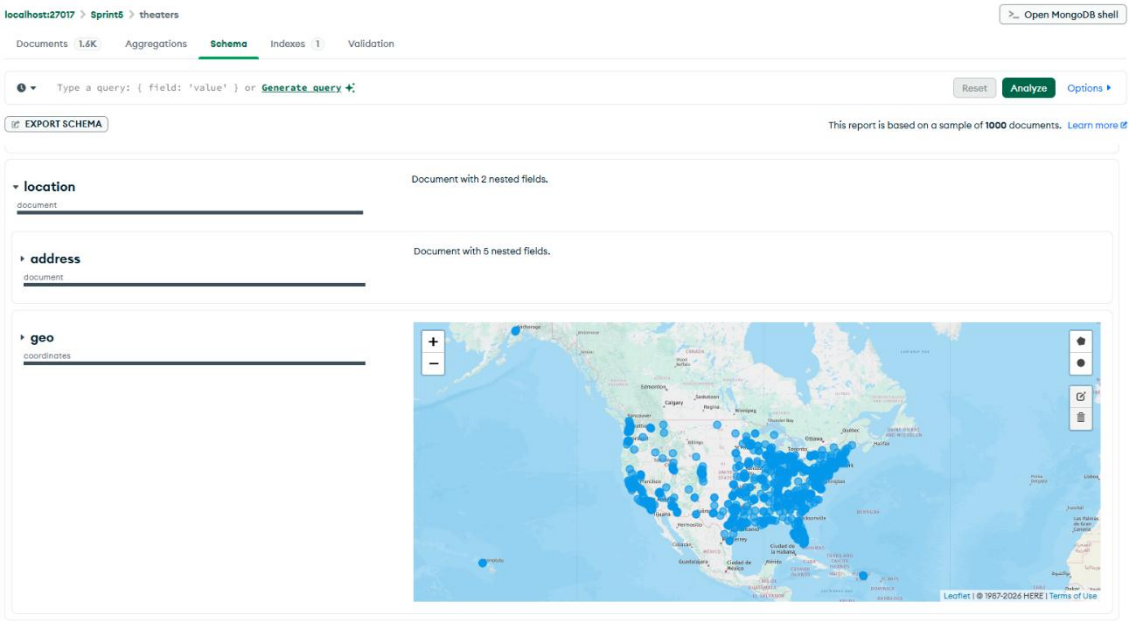
En el Compass fem un **Find**, buscant en el vector "**directors**" a "John Landis" i en el subdocument **imdb.rating** filtrem els que tinguin les notes entre 7,5 i 8 fent **$gte: 7.5 i $lte: 8**. El resultat és 4.




## Exercici 2

**Mostra en un mapa la ubicació de tots els teatres de la base de dades.**

En el mateix Compass, anem a la col·lecció "**theaters**" i d'aquí anem a la pestanya "Schema" i cliquem a "Analize Schema". En el subcamp **location.geo** trobarem el mapa de localitzacions. El mapa es pot generar ja que les dades de la col·lecció segueixen l'standard **GeoJSON** del tipus:


localhost:27017 > Sprint5 > theaters
Open MongoDB shell
Documents 1.6K
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
Reset
Analyze
Options
EXPORT SCHEMA
This report is based on a sample of 1000 documents. Learn more
location
document
Document with 2 nested fields.
address
document
Document with 5 nested fields.
geo
coordinates
Leaflet | © 1987-2026 HERE | Terms of Use